Project Interim Report
On

Library Management System
(Using Core Java & OOP Concepts)

RU-100-01-00012
Object Oriented Programming with Java

SESSION: 2025-26

Submitted By
Student Name - Harshit *Kumar*
Reg no - RU-25-10562

Bachelor of Technology in Computer Science & Engineering in association with Google

Under the Guidance of
Mr. Soumik karmakar

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI, CG

(April 2026)

Abstract

The Library Management System is a simple console-based Java application developed to perform basic library operations such as searching, issuing, and returning books. It is designed using object-oriented programming principles like encapsulation and modularity, ensuring a clear and organized structure. The system consists of three main classes: Book, Library, and Project4. The Book class stores details such as title, author, and availability status, along with methods to issue, return, and display book information. The Library class manages a collection of books using an array and provides functionalities to search for a book by title, issue a book if available, return a book, and display all books. The Project4 class contains the main method and implements a menu-driven interface using the Scanner class to interact with the user. This project demonstrates the use of core Java concepts such as classes, objects, arrays, loops, and conditional statements, and serves as a basic model that can be further enhanced with advanced features like databases and graphical user interfaces.

Introduction

The Library Management System is a basic Java application developed to simplify and automate common library operations such as searching, issuing, and returning books. In traditional systems, managing book records manually can be time-consuming and prone to errors, especially when handling multiple users and books. This project provides a simple and efficient solution by using a console-based interface that allows users to interact with the system easily.

The application is built using object-oriented programming concepts, where different classes are used to represent real-world entities like books and the library. The Book class stores details of each book, while the Library class manages a collection of books and performs various operations on them. The main class, Project4, controls the program flow and provides a menu-driven interface for user interaction.

This project helps in understanding core Java concepts such as classes, objects, arrays, loops, and conditional statements. It serves as a beginner-level model of a library system and lays the foundation for developing more advanced applications with features like databases, user authentication, and graphical user interfaces.

Objectives

- To develop a simple Library Management System using Java
- To implement object-oriented programming concepts such as classes and objects
- To apply encapsulation by using private data members and public methods
- To manage a collection of books using arrays
- To provide functionality to search books by title
- To enable users to issue books if they are available
- To allow users to return issued books
- To display all books with their availability status
- To create a menu-driven program for user interaction using Scanner
- To practice core Java concepts like loops, conditionals, and methods
- To build a foundation for developing real-world applications in the future

Scope

The scope of this Library Management System project is limited to performing basic library operations in a simple and efficient manner using a console-based interface. It allows users to view available books, search for books by title, issue books if they are available, and return issued books. The system manages a small collection of books using arrays and demonstrates fundamental object-oriented programming concepts in Java.
This project is intended for educational purposes and serves as a beginner-level model. It does not include advanced features such as database integration, user authentication, fine calculation, or graphical user interfaces. However, the system can be further expanded in the future by adding these features, making it suitable for real-world applications.

System Requirements

Hardware Requirements :

- Processor: Intel Core i3 or above
- RAM: Minimum 4 GB
- Hard Disk: At least 500 MB free space
- Input Devices: Keyboard and Mouse
- Display: Monitor with standard resolution

Software Requiremnets :

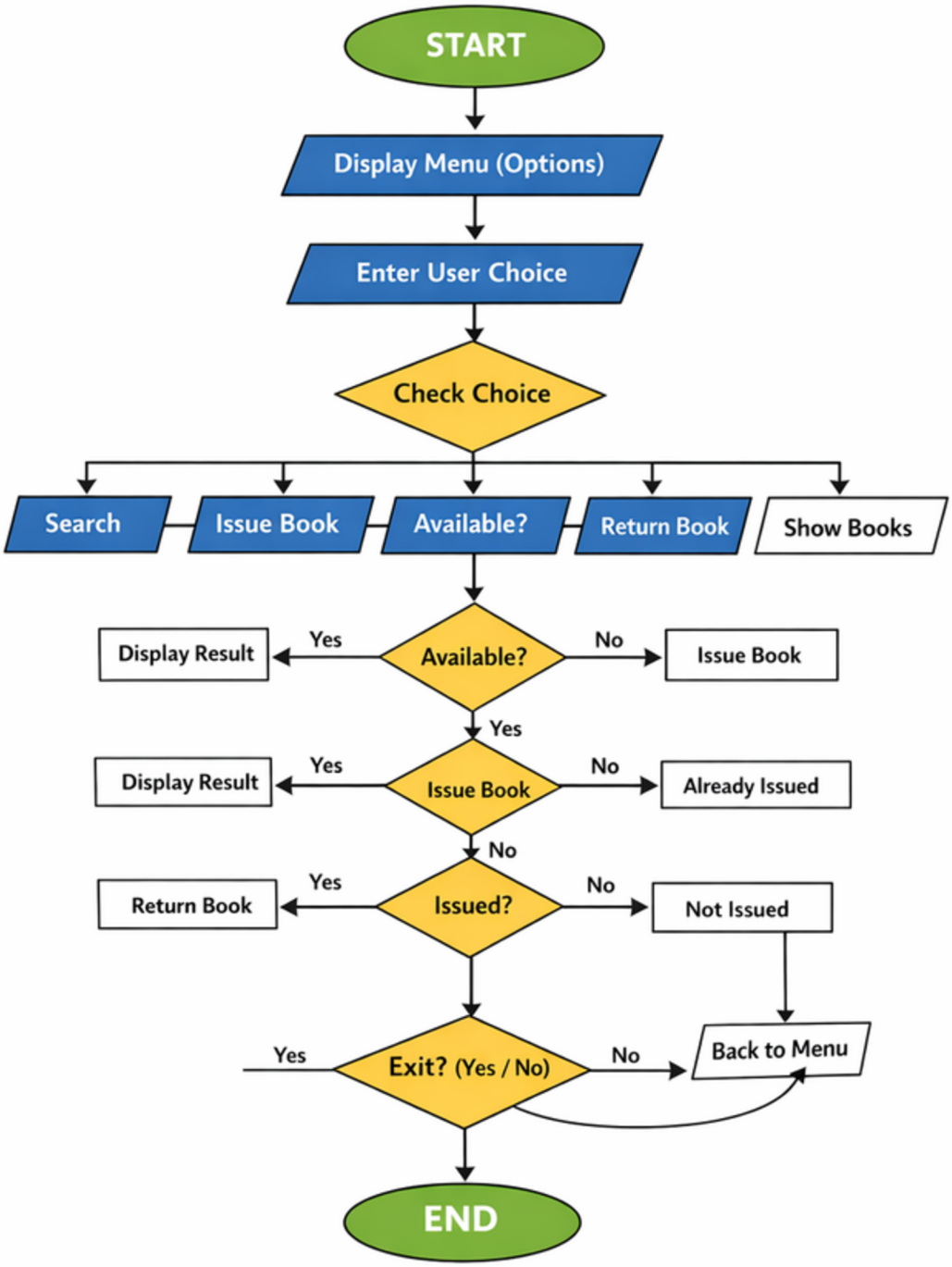
- Operating System: Windows / Linux / macOS
- Programming Language: Java (JDK 8 or above)
- IDE/Editor: Eclipse / IntelliJ IDEA / NetBeans / VS Code
- Compiler: Java Compiler (javac)
- Runtime Environment: Java Runtime Environment (JRE)

Methodology

Step 1 — **Input:** Book title is entered by the user through a menu-driven interface using the Scanner class.
Step 2 — **Object Creation:** Book objects are created in the Library class with title, author, and default availability status (true) using encapsulation.
Step 3 — **Menu Selection:** User selects an option (show, search, issue, return, exit) which is processed in the main class using switch-case.
Step 4 — **Search Operation:** searchBook(title) compares input with stored book titles using equalsIgnoreCase() and displays book details if found.
Step 5 — **Issue Operation:** issueBook(title) checks availability using isAvailable(); if true, issue() is called and status is updated to false.
Step 6 — **Return Operation:** returnBook(title) verifies issued status; if false, returnBook() is called and status is updated to true.
Step 7 — **Display Operation:** showBooks() iterates through the Book array and displays all books with their current availability status.
Step 8 — **Control Flow:** The system runs in a loop until the user selects exit option, ensuring continuous interaction.

Flowchart & Class Diagram

Flowchart



Class Diagram

Book	Library	Project4
Attributes	Attributes	Attributes
- title : String - author : String - isAvailable : boolean	- books : Book[]	-
Methods	Methods	Methods
+ Book(title, author)	+ Library()	+ main() : void
+ getTitle() : String	+ searchBook(title : String) : void	
+ isAvailable() : boolean	+ issueBook(title : String) : void	
+ issue() : void	+ returnBook(title : String) : void	
+ returnBook() : void	+ showBooks() : void	
+ display() : void		

Implementation

Step 1 — **Class Implementation:** A Book class is created with private data members (title, author, isAvailable) and public methods to ensure encapsulation.
Step 2 — **Constructor Implementation:** The constructor initializes book details and sets default availability to true.
Step 3 — **Method Implementation:** Methods like issue(), returnBook(), and display() are implemented to manage book status and information.
Step 4 — **Library Setup:** A Library class is implemented containing an array of Book objects initialized with sample data.
Step 5 — **Search Logic:** searchBook() method is implemented using a loop and equalsIgnoreCase() to find books by title.
Step 6 — **Transaction Logic:** issueBook() and returnBook() methods are implemented to update book availability based on conditions.
Step 7 — **Display Logic:** showBooks() method is implemented to iterate through all books and display their details.
Step 8 — **User Interface:** The main() method is implemented with a menu-driven system using Scanner, switch-case, and looping for continuous user interaction.

Sample Input / Output

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                                    Code

cd "/Users/harshitkumar/Desktop/PROJECT JAVA/" && javac project4.java && java project4
harshitkumar@Harshits-MacBook-Air PROJECT JAVA % cd "/Users/harshitkumar/Desktop/PROJECT JAVA/" && javac project4.java && java project4

📚 LIBRARY MANAGEMENT SYSTEM
1. Show Books
2. Search Book
3. Issue Book
4. Return Book
5. Exit
Enter your choice: 1

Available Books:
1. Java Basics by James Gosling - Available
2. Data Structures by Mark Weiss - Available
3. Python Fundamentals by Guido van Rossum - Available

📚 LIBRARY MANAGEMENT SYSTEM
1. Show Books
2. Search Book
3. Issue Book
4. Return Book
5. Exit
Enter your choice: 2
Enter book title to search: Java Basics
Book Found:
Java Basics by James Gosling - Available

📚 LIBRARY MANAGEMENT SYSTEM
1. Show Books
2. Search Book
3. Issue Book
4. Return Book
5. Exit
Enter your choice: 3
Enter book title to issue: Java Basics
Book issued successfully

📚 LIBRARY MANAGEMENT SYSTEM
1. Show Books
2. Search Book
3. Issue Book
4. Return Book
5. Exit
Enter your choice: 4
Enter book title to return: Java Basics
Book returned successfully

📚 LIBRARY MANAGEMENT SYSTEM
1. Show Books
2. Search Book
3. Issue Book
4. Return Book
5. Exit
Enter your choice: 5
Exiting...
harshitkumar@Harshits-MacBook-Air PROJECT JAVA %
```

Future Enhancements

Step 1 — **Database Integration:** Replace the array with a database (MySQL/SQLite) to store and manage large numbers of books permanently.
Step 2 — **User Management:** Add user accounts with login functionality for students and admin roles.
Step 3 — **GUI Development:** Implement a graphical user interface using Java Swing or JavaFX for better user experience.
Step 4 — **Book Addition/Deletion:** Provide options to add new books and remove existing books dynamically.
Step 5 — **Due Date System:** Introduce issue date and return deadline with fine calculation for late returns.
Step 6 — **Search Improvements:** Enable searching by author name, category, or multiple keywords.
Step 7 — **File Handling:** Store and retrieve book data using file handling instead of temporary memory.
Step 8 — **Notifications:** Add alerts or messages for due dates and availability of books.

Gantt Chart

1.Planning & Structure

The planning and structure of the Library Management System project begin with identifying key requirements such as searching, issuing, returning, and displaying books. The system is designed using an object-oriented approach, dividing the program into three main classes: Book for handling data, Library for managing logic, and the Main class for user interaction. An array is chosen as the data structure to store book objects due to its simplicity. Methods like searchBook(), issueBook(), returnBook(), and showBooks() are carefully planned to perform specific tasks efficiently.

The user interface is designed as a menu-driven system to make interaction simple and user-friendly. The control flow is managed using loops and switch-case statements, allowing continuous execution until the user exits. Proper input handling is done using the Scanner class to avoid errors. Overall, the structured design ensures clarity, easy maintenance, and the possibility of adding more features in the future.

2.Time Management

S. No	Task	Start	End
1	Requirement Analysis	Week 1	Week 1
2	System Design	Week 1	Week 2
3	Coding	Week 2	Week 2
4	Function Implementation	Week 2	Week 3
5	Testing	Week 3	Week 3
6	Debugging	Week 3	Week 4
7	Documentation	Week 4	Week 4
8	Final Review	Week 4	Week 4

Conclusion

The Library Management System project demonstrates the effective use of Java and object-oriented programming concepts to manage basic library operations. It provides a simple and user-friendly menu-driven interface that allows users to search, issue, return, and display books efficiently. The use of classes such as Book, Library, and Main ensures proper organization, modularity, and code reusability. Encapsulation is applied to protect data and maintain controlled access through methods.

The project successfully achieves its objective of automating basic library functions using arrays, loops, and conditional statements. Although it works on a small scale with in-memory storage, it provides a strong foundation for understanding system design and Java programming. In the future, it can be enhanced by adding database connectivity, graphical user interfaces, and user authentication to make it more advanced and suitable for real-world applications.

References

Books

1. **Head First Java –** This book was used to understand core Java concepts in a simple and practical way, especially object-oriented programming principles such as encapsulation, classes, and objects.
2. **Java: The Complete Reference –** This reference book provided in-depth knowledge of Java syntax, libraries, and advanced programming concepts required for building structured applications.

Websites

1. **GeeksforGeeks –** Used for learning Java programming concepts, problem-solving techniques, and practical implementation examples of object-oriented programs.
2. **W3Schools –** Provided quick reference materials for Java syntax, loops, arrays, and basic programming logic.
3. **Oracle –** Official Java documentation used for understanding standard libraries such as Scanner, String methods, and core API references.

Journal Articles

1. Research articles on “Object-Oriented Programming in Java” published in academic journals were referred to for understanding software design principles, modular programming, and real-world application development using Java.

Conference Papers

1. Papers presented in software engineering and computer science conferences on topics like “Efficient Design of Library Management Systems using OOP” and “Automation in Academic Libraries using Java-based Systems” were reviewed to understand modern approaches to system design and improvement techniques.